CSE2530 COMPUTATIONAL INTELLIGENCE

Assignment 2: ACO & GA

February 23, 2026

0. INTRODUCTION

Our goal in this assignment is to enable a robot to navigate in a supermarket and find multiple products. The supermarket will be modeled as a labyrinth (further referred to as "maze"). In order to save time and energy, your bot should determine the fastest route between a set of locations to visit within the supermarket. This problem is usually referred to as the *Traveling Salesman Problem* (TSP) and can be solved with a genetic algorithm.

Before it can determine the fastest way to visit all locations, your robot should have an idea of what the distance is between these locations. To achieve this, you will implement a swarm intelligence approach. Specifically, path finding based on Ant Colony Optimization (ACO).

Hence this assignment is split into 2 sub-problems: TSP and path finding. You will hand-in the report with your implementation and answers to all questions. You can work through this assignment in the order detailed below. To help you get started, we provide you with a Python template for the solution; information about the template is included in Appendix B.

MAZE VISUALIZER

For this assignment, you can test your mazes using the visualizer included in the folder with code provided in a Jupyter Notebook format. This tool allows you to verify whether your answer seems reasonable by loading in a maze along with the route.

DELIVERABLES

You will submit a report in the Jupyter Notebook format, consisting of your implementation and your answers the questions from Sections 1, 2, and 3. **You should hand in the final working version of your implementation on Brightspace (as .ipynb file)**. Your notebook must (1) use ACO to find the best path in the hard maze, and (2) execute the genetic algorithm using the data from the provided file. Check if the notebook works as expected before pressing the `Submit` button!

Furthermore, we ask you to **deliver the following files via Brightspace**:

- route files for the `Easy`, `Medium`, and `Hard` mazes (the `Insane` maze is not required):

  `"<group>_easy.txt"`

  `"<group>_medium.txt"`

  `"<group>_hard.txt"`

- a final action file for Part 1 (see route syntax in Section 1):

  `"<group>_actions_TSP.txt"`

Here, `<group>` is the number of your group on Brightspace.

You will use Python for this assignment. You are also allowed to use NumPy as a linear algebra library – **this is the only library that you can use to build the algorithms**. You may however use other libraries for tasks unrelated to the functionality of your Ant Colony Optimization and Genetic Algorithm (for instance for plotting).

Note: Your implementation will only be graded from 0 to 6 points (see rubric in Section C). **Do not rely on the code to support your answers for questions in Sections 1, 2, and 3.**

We ask you to deliver your work exactly as explained above. **If (a) your files are not in the correct format or (b) the main notebook does not run, your work will not be graded.** We provide you with a Python environment with these libraries included (setup instructions on Brightspace). We require you to use that environment specifically to avoid compatibility issues.

Fraud will not be tolerated. You are allowed to discuss concepts and ideas with colleagues from other groups, but you are not allowed to share code outside your own group. The same applies to submissions from previous years. You may ask questions on Answers-EWI (while keeping in mind that it's a public forum, so please don't share partial solutions), or to a TA during the labs.

USE OF AI TOOLS

For transparency, each submitted assignment must include a short statement describing whether and how AI-based tools were used during the preparation of the work. Where ap-

plicable, brief examples of how AI tools were used are welcome. This statement must be included as part of the submitted report.

If we believe that AI was used beyond what is stated, or that the submitted work is not the student's own, we reserve the right to request an in-person explanation of the submission to verify the student's understanding.

The deadline for the assignment is Monday **23rd of March 2025 at 23:59**.

1. Part 1: The Traveling Robot Problem

In the first part of the assignment you will be tackling a modified version of a classic AI problem, commonly known as The Traveling Salesman Problem (TSP).

As you can imagine (and probably know from your own experience), there are a lot of possible routes to take through a supermarket if you want to pick up several products. For 3 products, there would only be $3 \times 2 \times 1 = 6$ possible routes but this number increases exponentially. For 20 products, we already have $20! = 2.4 \times 10^8$ possible routes! Clearly, brute-force trying out all these routes to find the fastest would take too much computational time. Instead, you will be implementing an intelligent way of converging towards a (nearly) optimal path.

Getting Started

On Brightspace you will find the data needed for this part of the assignment:

- the `Hard` maze (you can read about its representation in Appendix A);
- the file containing optimal paths between all pairs of products (your input);
- the x and y coordinates of a list of products, which your robot must go and pick up;
- the route visualizer (*optionally*).

Route Syntax for Part 1

You need to encode your results (routes) by their length, starting location, and a sequence of actions to be taken by the robot. The header of the file representing the total route length, is stored as an integer. This value is equal to the number of actions. A "`;`" should follow this value. The starting point should be given by two number: the x-coordinate and the y-coordinate, separated by a comma ("`,`"). A semicolon ("`;`") should follow the y-coordinate.

There are five possible actions: steps `East`, `North`, `West` or `South`, and `take product`. The first four actions are encoded using an integer;

- 0 = East
- 1 = North
- 2 = West
- 3 = South

Within the steps encoding, all characters other than $\{0, 1, 2, 3\}$ are invalid. Again, each action should be followed by a semicolon ("`;`"). You can test the validity of the route file with the visualizer. The provided templates will put the files in the right format for you.

The last action indicates when a product has to be picked up by the robot. This is done by inserting the line `take product #<Product Number>;` in between the step actions. Part of a valid result file may thus look like:

```
83;
0, 1;
3;
3;
0;
take product #6;
3;
2;
2;
take product #15;
3;
1;
1;
1;
take product #4;
1;
0;
(...)
```

The provided templates contain export functions to get the appropriate file format needed.

1.1. Problem Analysis

To get a good understanding of the nature of our problem, start by analyzing its characteristics. Your robot will have to pick up a number of different products from all over the supermarket, which can be found at certain locations.

Question 1: (1 pt.)
Look into the literature and explain how the TSP problem is usually defined.

Rubric 1:

- (1 pt.) Correct definition of a TSP problem with references.

Question 2: (3 pt.)
In a classic TSP, all cities (nodes) are singularly connected to all other nodes and relative distances are known and symmetric (a weighted complete graph). Give three ways in which our problem is different.

Rubric 2:
- (3 × 1 pt.) Distinct, correct, and meaningful difference.

Question 3: (2 pt.)
Why are computational intelligence techniques appropriate to solve the TSP? What are the characteristics which let us to tackle typically intractable problems?

Rubric 3:
- (1 pt.) Sensible discussion of CI techniques in the context of TSP.
- (1 pt.) Sensible explanation of at least two characteristics.

1.2. Genetic Algorithm

A classical method of solving a TSP are genetic algorithms. This technique is part of the evolutionary computation methods. Biological evolution is sometimes described as non-random selection through random gene mutations. Genetic algorithms can be described using the same nomenclature: random search through non-random exploration of the solution space. Since this space is too big to verify all possible outcomes, we need an algorithm which directs us towards viable options. **Answer the questions based on your implementation**.

Question 4: (2 pt.)
What do the genes represent? How will you encode your chromosomes?

Rubric 4:
- (2 pt.) Fully correct representation of the problem.

Question 5: (1 pt.)
Which fitness function will you use? Why is this a suitable choice?

Rubric 5:
- (1 pt.) Sensible fitness function with an explanation.

Question 6: (1 pt.)
How are parents selected from the population?

Rubric 6:
- (1 pt.) Sensible parent selection algorithm with an explanation.

Question 7: (3 pt.)
What genetic operations did you implement? What are their functions (goals)?

Rubric 7:
- (2 pt.) At least two fully correct genetic operations.
- (1 pt.) Sensible discussion of their functions.

Question 8: (1 pt.)
How do you prevent local minima?

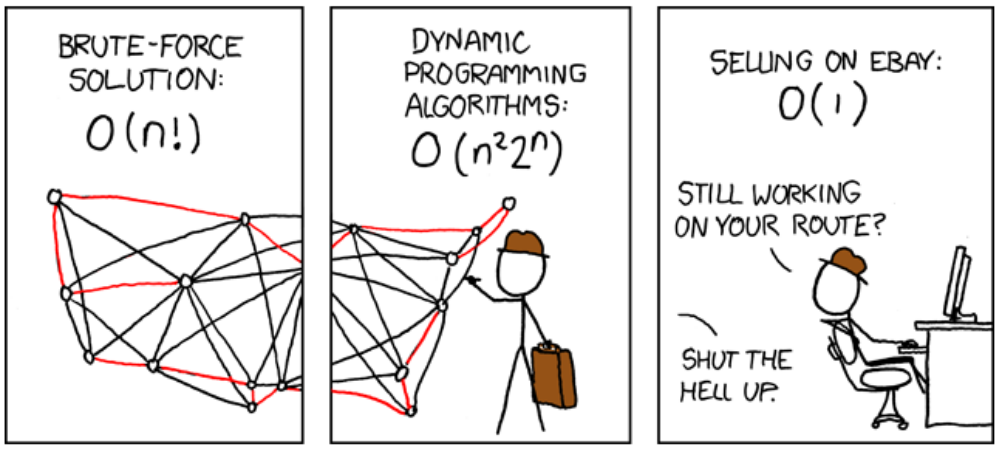
Rubric 8:
- (1 pt.) Sensible approach to prevent local minima.

Question 9: (2 pt.)
What is elitism? Have you applied it? Why (not)?

Rubric 9:
- (1 pt.) Correct definition of elitism.
- (1 pt.) Sensible discussion why it is (not) needed.

Question 10: (3 pt.)
Apply your algorithm to find the shortest route using the provided data. What is the length of your solution? Is it optimal?

Rubric 10:
- (2 pt.) Correctly applied the genetic algorithm on the provided data.
- (1 pt.) Sensible discussion of the solution.



source: http://xkcd.com/399/

2. Part 2: Path Finding Through Ant Colony Optimization

In the second part of this assignment you will teach your robot how to move through a maze and implement a method for finding a quick route between a specified start and end point. The goal of the assignment is to use the methods learned in this course to tackle the problem, hence **you are not allowed to use search algorithms like Dijkstra or A***.

2.1. Route Syntax for Part 2

The route syntax is the same as for Part 1; however, you will not be making use of the `take product` actions. In other words, the only available actions are `East`, `North`, `West` or `South`.

2.2. Observing The Problem

Question 11: (1 pt.)
What is the purpose of Ant Colony Optimization? In what settings is it typically used?

Rubric 11:
- (1 pt.) Sensible description of ACO.

Question 12: (1 pt.)
Make a list of the "topographical" features you can expect in a maze that increase the difficulty of finding the finish line and require creative solutions. Discuss **at least 2**.

Rubric 12:
- (1 pt.) Sensible discussion of at least 2 features.

Question 13: (2 pt.)
Give an equation for the amount of pheromone dropped by the ants. Explain why ants need to drop the pheromones in the maze.

Rubric 13:
- (1 pt.) Fully correct equation with discussion of variables.
- (1 pt.) Sensible explanation why pheromones are needed.

Question 14: (2 pt.)
Give an equation for the evaporation; it should contain variables which you can use to optimize your algorithm. How much pheromone will evaporate in every iteration? What is the purpose of pheromone evaporation?

Rubric 14:
- (1 pt.) Fully correct equation with discussion of variables.
- (1 pt.) Sensible explanation why evaporation is needed.

2.3. Implementing the Ant Algorithm

Now, implement the **standard** ant algorithm. The output of your algorithm should be a route between the given start and end point. You may want to already take the route syntax into account. Your algorithm should satisfy a few conditions:

I. At least the following parameters should be **easily modifiable** (as hyper-parameters):

   a) maximum number of iterations (to prevent infinite loops);

   b) number of ants patrolling in each iteration;

   c) amount of pheromone dropped by ants;

   d) evaporation parameters;

   e) convergence criterion.

II. The start and end points of the route should be variable. Your code should be able to find a short route between arbitrary two points, provided that such route exists.

At this point it is sufficient to assume that the maze does not have any of the hard features you considered in Question 11. Simply implement the standard ant algorithm. You will update your algorithm later to deal with the harder features but it is useful to already test how the ants behave. Still, its good programming practice to have the future upgrades in the back of your mind while writing your basic algorithm to facilitate later adjustments.

2.4. Upgrading Your Ants with Intelligence

You will now adapt the ant algorithm to deal with the difficult features that you came up with. There are many possible approaches here – it is a good idea to create small test mazes to see how your code behaves when these features occur.

Note: We expect you to keep with the biologically-inspired nature of the algorithm. When designing your improvements, think about the actual behaviour of animals such as ants. Some adaptations will be considered "cheating" within the scope of this assignment. For instance, when your ant enters the maze it should not know where the exit is located, so any form of

directed search will not be accepted. However, it is accepted if your ants use some form of memory. If you are in doubt about your improvements, consult the TAs in the lab.

Question 15: (5 pt.)
Improve the ant algorithm using your own insight. Explain which problems you are tackling and how? We would like to see at least two **meaningful** improvements (as an example you may give your ants some form of memory). Use visual aids to show that each of your modification results in meaningful improvement.

Rubric 15:
- (2 × 2 pt.) Distinct and sensible improvement with proper explanation.
- (1 pt.) Informative visual aids showing a difference in results.

2.5. Parameter Optimization

We have provided 3 different grading mazes for you to work with: `Easy`, `Medium` and `Hard`. You are expected to run your code on each of these mazes to see how your code behaves for an increasing maze complexity. While doing so, you should tweak the algorithm parameters to make your code converge quickly.

The optimal set of parameters (evaporation constant, amount of pheromone dropped, etc.) depends strongly on the precise shape of the maze. A bigger maze will typically have different optimal parameters, but so will a maze with other features (e.g., many open areas).

Question 16: (4 pt.)
Your task is to find a good set of parameters for each of the grading mazes. You may do so by varying the parameters and subsequently running your algorithm. Report your approach to tuning the parameters. Assist your text with graphs showing the relationships between the parameters and the speed of convergence. Present the impact of the hyper-parameter settings on the performance with visual aids.

Rubric 16:
- (1 pt.) Clearly reported hyper-parameter settings for each maze.
- (1 pt.) Sensible tuning strategy with proper explanation.
- (1 pt.) Informative visual aids.
- (1 pt.) Sensible discussion of the impact of hyper-parameters on performance.

Question 17: (2 pt.)
Using your answer to the previous question, how does the complexity and size of a maze influence the "optimal" value of each hyper-parameter?

Rubric 17:
- (1 pt.) Sensible observations for all five hyper-parameters.
- (1 pt.) Sensible explanation of these observations.

2.6. The Final Route

Question 18: (0 pt.)
Run your code using your decent set of parameters on each of the grading mazes. Output your route as described in the Route Syntax section (you can verify the correctness by opening your route in the visualizer). See also "Deliverables" in Section 0.

Rubric 18:
- Working code is a knock-out criterion

Run your code using your decent set of parameters on each of the grading mazes. Output your route as described in the Route Syntax section (you can verify the correctness by opening your route in the visualizer). See also "Deliverables" in Section 0.

2.7. Synthesis

We will now combine the two parts of the assignment. Use your Ant Colony Optimization algorithm in the `TSPData.py` file to generate your own estimate of distances between the start and each product, each pair of products, and each product and the end. Use this file to re-run your genetic algorithm and find the best order of actions.

Question 19: (2 pt.)
What are the differences (length, order of items, etc.) between the current path and the path found by your algorithm in Part 1? Which solution is better? Why?

Rubric 19:
- (1 pt.) Sensible discussion of the differences between paths.
- (1 pt.) Correct observation which solution is better with explanation.

3. Open Questions

3.1. Reflection

For this part of the assignment, you should think of genetic algorithm approaches in general, rather than in the context of your specific implementation.

Question 20: (2 pt.)
Genetic algorithms need a fitness function but there are many scenarios where you will not know the correct (underlying) fitness function. Discuss an example where this would be the case. How can it cause problems?

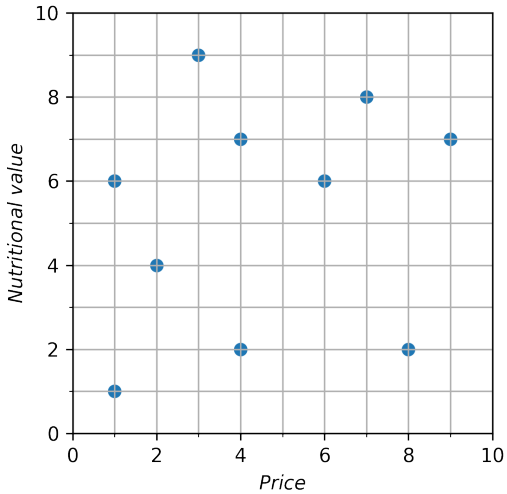
Rubric 20:
- (1 pt.) Sensible example with proper discussion.
- (1 pt.) Correct characterization of resulting problems.

Question 21: (2 pt.)
What about "survival functions"? Do we always want the strongest individual to survive? How can we mitigate these problems through the mechanisms of selection?

Rubric 21:
- (1 pt.) Sensible discussion why strongest individuals should (not) survive.
- (1 pt.) Sensible explanation of at least two mitigation strategies.

3.2. Pen and paper

Finally, we have a question for you to be solved on paper that serves as the preparation for the final exam. It is independent from the rest of the questions in this document. You are asked to submit only answer but we strongly suggest that each team member solves Q22 separately, so that all of you can practice with the calculations.



Question 22: (4 pt.)
Imagine that our robot attempts to select the best set of foods based on their price and nutritional value (see the plot above). Which of the variables do you think we should maximize or minimize? What are the Pareto frontiers? If the robot can carry only four products, which of them will it choose according to the NDSGA-II algorithm? Justify all of your answers.

Rubric 22:

- (1 pt.) Correct characterization of the problem.
- (2 pt.) Correct identification of all Pareto frontiers.
- (1 pt.) Correct final answer with justification.

Question 23: (0 pt.)
Finally, we ask you to indicate for each group member the approximate contribution **in terms of effort** for each of the criteria in Table 3.1 below. For example, if all four group members contributed equally to the writing of the final report, you should indicate 25% for each of them in the *Report (original draft)* component.

This information will not be directly considered during the grading, but we will take it into account in case any conflicts arise. **Be honest about your own contributions and make sure as a group that everyone agrees about the table included in your final report**. Also, remember that you can always – individually or as a group – reach us at *ci-cs-ewi@tudelft.nl*.

Component	Name A	Name B	Name C	Name D
Code (design)	A	B	C	D
Code (implementation)	A	B	C	D
Code (validation)	A	B	C	D
Experiments (execution)	A	B	C	D
Experiments (analysis)	A	B	C	D
Experiments (visualizations)	A	B	C	D
Report (original draft)	A	B	C	D
Report (reviewing and editing)	A	B	C	D

Table 3.1: *Distribution of work*

A. Grading Mazes

Besides creating, analyzing, and testing mazes yourself, you will need to run your code on each of our grading mazes. These mazes are available on Brightspace. There are 5 mazes available: `Toy`, `Easy`, `Medium`, `Hard` & `Insane`. Note that it's not mandatory to solve the `Insane` maze, but you can use it to show off the efficiency of your code.

The mazes are ASCII encoded matrices of 1s (accessible) and 0s (inaccessible). Your robot is allowed to walk single discrete steps within this matrix; however, only the tiles that are accessible. For your convenience, the maze files start with two numbers which indicate the width and height of the matrix: a 10 by 20 matrix – 10 rows by 20 columns – is indicated as "20 10". You can open the maze files in any text editor to see what they look like.

```
20 10
1 0 1 1 1 1 1 1 1 1 0 0 0 1 0 1 0 1 1 1
1 0 1 1 1 1 0 1 0 1 1 1 0 1 0 1 0 1 0 1
1 0 1 1 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
1 1 1 0 0 1 0 1 0 1 0 1 0 1 1 1 1 1 0 1
0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 1 0 1 1 1
1 1 1 1 1 1 0 1 0 1 0 0 1 0 0 1 0 0 0 1
1 0 1 0 1 0 0 1 0 1 0 0 1 0 0 1 0 1 1 1
1 1 1 0 1 0 0 1 1 1 0 1 1 0 1 1 1 1 0 1
0 1 0 0 1 1 1 1 0 1 1 1 0 0 1 0 0 1 0 1
0 1 1 1 1 0 0 1 1 1 0 1 1 1 1 0 0 1 1 1
```

Accompanied with each grading maze is a coordinate file. This file consists of 4 numbers (2 sets of coordinates). The first two integers correspond to the starting position in the maze: "x, y;". Note that these are x and y coordinates, not a row & column number! The second two integers correspond to the ending point of the maze. Hence for a 50x50 maze, starting at the top left and finishing in the bottom right, the coordinate file would read:

```
0, 0;
49,49;
```

B. Instructions for template

We provide you with a Python template to get started. There are a number of stubs/unimplemented functions that you will need to implement to solve the assignment. You can create extra functions/modify existing functions as you see fit. The classes `AntColonyOptimization`, `GeneticAlgorithm` and `TSPData` have a main function that can be executed.

We also provide you with an overview of the classes available in the template.

B.1. Part 1

GeneticAlgorithm – driver function for the Genetic Algorithm. Takes a file that contains a 2D-matrix containing the routes between products and attempts to optimise those. You will need to implement the genetic algorithm yourself.

TSPData – builds and stores the data for all the shortest paths between the products. You **build** the data file only after completing Part 2 of the assignment based on your implementation of the AntColonyOptimzation. Before then, you will use the provided file (`optimal_tsp`) to work on your genetic algorithm in Part 1. You also have to tune the parameters. TSPData will write the final action file given a solution to the TSP problem.

B.2. Part 2

StandardAnt, IntelligentAnt – represent the ants finding a path through the maze. You will need to implement logic how an ant finds a route through the maze.

AntColonyOptimization – driver function that finds an optimal route between two points. Allows you to tune the parameters of the algorithm. You will need to implement the creation of a maze, the ants to allow them to run through the maze.

Coordinate – represents a coordinate. All functions are implemented.

Direction – represents the directions an Ant can take. All functions are implemented.

Maze – represents a maze. You will need to implement the initialization, addition and reset of the pheromones. Besides this you will need to implement a function that returns SurroundingPheromone for a given coordinate.

PathSpecification – represents the pair of coordinates that indicate the start and end points of a route. Used as initialization for shortest paths/ant colony optimzation. All functions are implemented.

Route – represents a route. All functions are implemented.

SurroundingPheromone – represents the surrounding pheromone for a given coordinate in the maze. All functions are implemented.

C. General rubric

Example calculation for criterion *1. Quality of the report*: if your report is (1) *mostly structured*, it has (2) *clear, well-reasoned answers* and (3) *clear, in-depth discussions*, but it contains (4) *many spelling and grammar mistakes* you would receive for this criterion:

$$3 \cdot 0.25 + 6 \cdot 0.25 + 6 \cdot 0.25 + 0 \cdot 0.25 = 3.75 \text{ out of 6 points.}$$

Criterion	0 pt.	3 pt.	6 pt.
1. Quality of the report (each sub-criterion 25%)	Report not structured Many unclear answers Little to no discussion Many spelling/grammar mistakes	Report mostly structured Mostly clear answers Mostly correct discussions Some mistakes	Report properly structured Clear, well-reasoned answers Clear, in-depth discussions Almost no mistakes
2. Quality of the code (each sub-criterion 25%)	Code not structured Improper formatting Little to no comments Many implementation mistakes	Code mostly structured Mostly correct formatting Most objects have comments Some mistakes	Code fully structured Clear, proper formatting Clear, thorough comments Almost no mistakes
3. Scientific method (each sub-criterion 20%)	Approach not adequate Work not sound or reliable Code not tested Results not verified Missing/incoherent arguments	Approach mostly adequate Work mostly sound and reliable – – Mostly coherent arguments	Approach fully adequate Work fully sound and reliable Code tested by unit tests Results mostly verified Coherent, concise arguments
4. Visual aids (each sub-criterion 33%)	Plot types not suitable Many plots are not readable Many missing elements	At most 1 unsuitable plot Clear, readable plots Plots have some missing elements	Plot types always suitable Clear, readable plots No missing elements; all plots have identifiers, clear captions, and references in text
5. Reproducibility (each sub-criterion 33%)	Results not reproducible No code for plots/tables Results have unclear provenance	Results mostly reproducible Code mostly available –	Results fully reproducible Code for plots/tables available Report follows experiments

To be obtained from Questions 1 - 22: 46 **points**
To be obtained from the general rubric: $5 \cdot 6 = 30$ **points**
Total: 76 **points**